¿El top-5 vende, o solo muestra lo que se vendería igual?

Retorno causal de un algoritmo de recomendación · TPI EAyMR 2026 · UCA

Verónica Ana López Marsico · Lucía Báez · Antonella Robiollio

18/11/2026

Prof. Ivan Gonzalo Vilaboa · Estadística Aplicada y Modelos de Regresión

1 · El problema de negocio

- Unos 25 hoteles por búsqueda; la atención se concentra en los primeros. **Los slots del top son escasos y de suma cero.**
- El recomendador pone arriba hoteles que se venderían igual: ¿el slot **causa** ventas o solo las **predice**?
- La respuesta fija el precio de un slot premium y a quién conviene dárselo.
- **Preguntas:** efecto causal del top-5 sobre la reserva · sesgo del dato del algoritmo · rol del clic · para qué hoteles rinde más.

2 · Datos y experimento natural

- Expedia ICDM 2013 (Kaggle): 9.917.530 filas, 399.344 búsquedas; muestra de 30.000 búsquedas completas.
- `random_bool`: parte de las búsquedas se mostró en **orden aleatorio** → benchmark para contrastar el supuesto Backdoor.
- Auditoría: centinelas, precio en dos escalas, slots fijos 5/11/17/23 fuera del dataset, dataset seleccionado sobre el clic (colisionador natural).
- Faltantes: MCAR se rechaza (Little: $d^2 = 75.952$, 73 gl). MAR → mediana por grupo **ajustada en Train** + indicador; MNAR estructural → no se imputa.

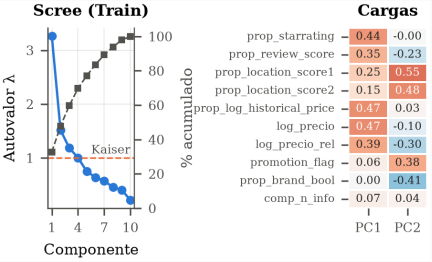
3 · Inferencia: el sesgo de posición existe

- Con orden aleatorio, la tasa de clic del top-5 es **2,16 veces** la del resto (t pareado, $d_z =$ 0,306; MDE $d_z =$ 0,023).
- ANOVA del log precio por estrellas: Levene rechaza → **Welch** $F = 1.807$, $\eta^2 = 0{,}254$; Kruskal y Games-Howell/Tukey/Bonferroni coinciden.
- Shapiro con n grande rechaza ante desvíos irrelevantes: miramos asimetría, curtosis y Q-Q.
- Gasto: recurrentes vs. nuevos, **no se rechaza H0** (MDE $g = 0{,}090$).



4 · Gauss-Markov y PCA

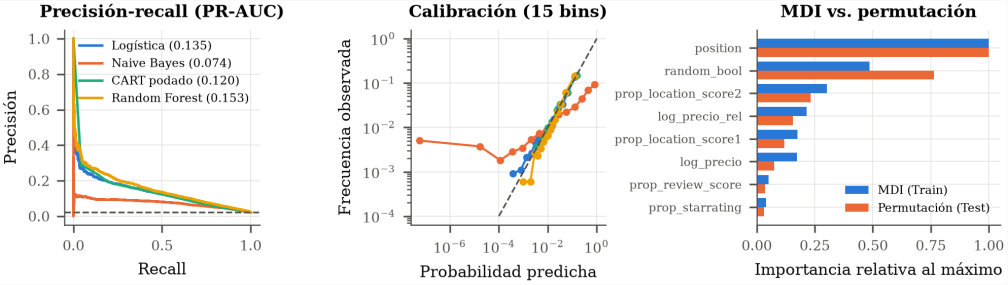
- MCO del log precio: $R^2 = 0{,}604$. Breusch-Pagan rechaza $\rightarrow$ **HC3**; colas pesadas (curtosis 5,8) $\rightarrow$ **Huber**; Durbin-Watson $= 1{,}95$.
- 748 influyentes ($D > 4/n$), ninguno con $D > 0,5$; VIF máx. 2,98.
- PCA **solo en Train**: PC1 = categoría-precio; PC2 = ubicación y promoción. Lo reservado se separa por PC2.



5 · Torneo predictivo

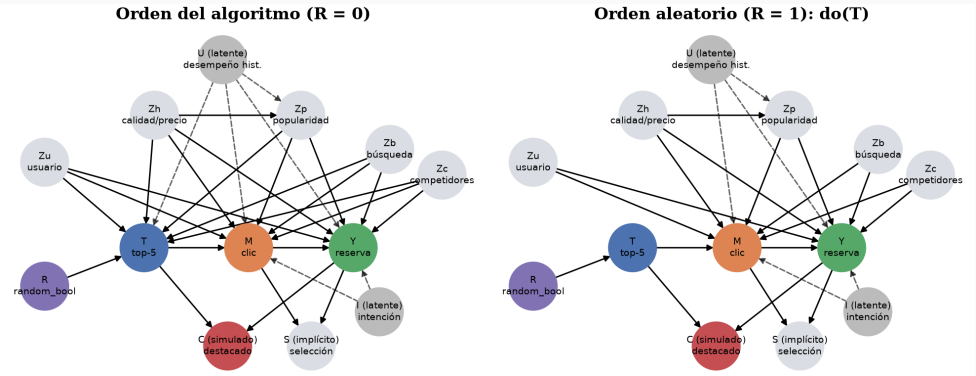
- Logística L1/L2, Naive Bayes, CART podado (α por CV) y Random Forest (5 hiperparámetros, OOB $\approx$ CV); CV agrupada por búsqueda; PR-AUC como métrica (prevalencia $\approx 2\,\%$).
- Ganador: **Random Forest**, PR-AUC 0,153, ROC-AUC 0,854, Brier 0,0196. Holdout cronológico: 0,144.
- Árbol sin poda: PR-AUC 1,00 en Train vs. 0,031 en Test $\rightarrow$ sobreajuste.
- Naive Bayes ordena (ROC-AUC 0,796) pero sus probabilidades no sirven: la independencia condicional se viola.

6 · Calibración y explicabilidad



- Reliability diagrams: logística, CART y RF calibrados; Naive Bayes empuja a 0 y 1.
- MDI favorece variables de alta cardinalidad ($\rho = 0{,}70$); la permutación en Test lo corrige. **La posición es la variable más predictiva.**

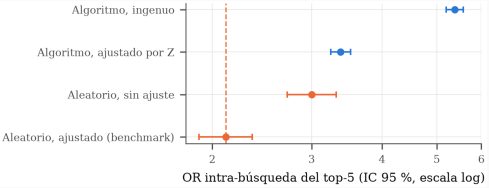
7 · DAG, identificación y positividad



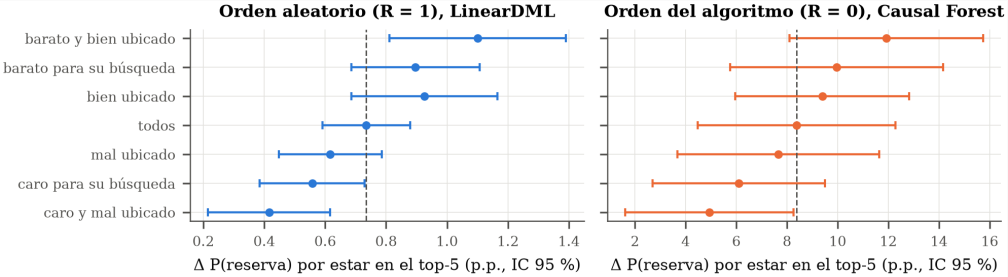
- Con el algoritmo, Z bloquea $T \leftarrow Z \rightarrow Y$ pero no $T \leftarrow U \rightarrow Y$. Con orden aleatorio, el conjunto vacío alcanza. Clic $=$ mediador; selección S y C $=$ colisionadores.
- Overlap: 0,010 % fuera de [0,01; 0,99]; IPW Hájek 0,0882 [0,0854; 0,0909], igual recortado.

8 · El efecto causal: observacional vs. cuasi-experimento

- OR intra-búsqueda (logit condicional): **benchmark 2,11** [1,90; 2,35], observacional ajustado 3,38, ingenuo 5,38.
- Ajustar por Z explica el 50 % del sesgo. **El ajuste por lo observable no alcanza**: cociente de OR 1,60 [1,42; 1,80] (bootstrap por búsqueda, 1000 réplicas); se sostiene en diferencia de riesgos y re-ponderando.
- Trampas: ajustar por el clic deja $\Delta p \approx 0{,}0042$; condicionar en C induce Berkson.



9 · CATE por segmento y refutaciones



- La heterogeneidad **se confirma con orden aleatorio** (LinearDML, R = 1): el efecto baja con el precio relativo y sube con la ubicación.
- Refutaciones (causa común aleatoria, placebo, subconjunto 80 %): las tres pasan. Miden estabilidad, no detectan U: eso lo hace el benchmark.

10 · Demo en vivo de la app

- **Diagnóstico + PCA:** filtros por estrato, distribuciones y proyección en PC1-PC2 (solo agregados).
- **What-Is:** probabilidad calibrada del Random Forest para un hotel nuevo, con explicación local.
- **What-If:** DAG, ATE, CATE según el perfil elegido y semáforo de las tres refutaciones.
- **Bonus:** copiloto con tool use sobre el motor causal y guardarraíl contra cifras inventadas.
- App: `<URL_APP>` (también en el README); plan B: contenedor Docker probado.

11 · Conclusiones y recomendaciones

- **El top-5 causa reservas, pero menos de lo que muestran los datos del algoritmo**: valuar el slot con el efecto causal (el dato del algoritmo sobrevalúa el OR en 60 %).
- **Asignar por segmento:** 11,0 reservas cada 1.000 exposiciones con hoteles baratos y bien ubicados vs. 4,2 con caros y mal ubicados; reasignar suma $\approx$ 1.530 USD cada 1.000.
- **Mantener tráfico con orden aleatorio** como auditoría permanente del ranking.
- La variable más predictiva **no es una palanca** de la misma magnitud.
- Límites: búsquedas con ≥ 1 clic, SUTVA (suma cero), aleatorización imperfecta, riesgo de licencia.

12 · Scrum, Quality Gate y bitácora

- Roles: V. A. López Marsico (Project Lead y Scrum Master) · L. Báez (Backend, ML y causal) · A. Robiollio (Frontend y QA).
- Un sprint por fase, PR por fase con veredicto de QA antes del merge a `main`: aprobamos las Fases 0-2 y bloqueamos las 3-5 hasta corregir sus defectos.
- Quality Gate en CI: `ruff`, tests de no-fuga (`pytest -m leakage`), suite completa y build de la imagen Docker.
- Bitácora: 5 entradas; **6 experimentos fallidos** documentados, entre ellos una fuga Train/Test en la imputación detectada en la auditoría de cierre y corregida.
- Uso de IA (Claude Code) trazado error por error, cada uno con el test que lo refuta.